Module A Report: ACID Validation, Transaction Management, and Crash Recovery for a B+ Tree Database Engine

Assignment Group
CS 432 Databases, Semester II (2025 to 2026)
Indian Institute of Technology Gandhinagar

***Abstract*—This report presents the complete design, implementation, and validation of Module A for Track 1 Assignment 3. The work extends an Assignment 2 in-memory B+ tree table abstraction into a crash-aware, transaction-capable mini database engine with explicit ACID behavior. The final implementation introduces a binary heap-file storage layer, write-ahead logging (WAL), multi-table transactions, recovery logic for crash scenarios, referential integrity with cascading actions, secondary indexing, condition-aware query execution, and join strategies that adapt to available indexes.**

Instead of reporting only pass/fail outcomes, this report explains what was built, why each component was introduced, and what behavior was observed under realistic usage and stress-style scripts. We compare the initial codebase and final codebase file-by-file, document transaction and recovery flows, and present execution evidence from usage scripts including rollback verification, persistence checks, deep cascade behavior, and concurrent update correctness checks.

***Index Terms*—B+ tree, ACID, transaction management, write-ahead logging, heap file, crash recovery, isolation, foreign key, secondary index, database systems.**

I. Submission Front Matter

GitHub Repository Link (required): https://github.com/zainabkapadia52/Multi-Course-Attendance-Management-Portal

Video Link (required): https://drive.google.com/file/d/1tYMcv9ku8V_6bmWq2w5MID89XogXjdmd/view?usp=sharing

II. Assignment Objective and Scope

The assignment requires extending the existing Module A database engine with robust transaction behavior and recovery support while preserving the B+ tree as the authoritative relational storage path. The required outcomes include:

- Atomicity for multi-relation operations.
- Consistency of records and constraints after every operation.
- Isolation during concurrent access.
- Durability across restart/crash conditions.
- Demonstration with at least three relations and cross-table updates.

The final codebase was therefore evaluated not as isolated data-structure functions, but as an integrated transactional storage engine.

III. Initial vs Final Codebase Evolution

A. Initial State (Assignment 2 Baseline)

The initial code in `Module-A/database` contained a compact in-memory architecture:

- `bplustree.py`: in-memory B+ tree with search, insert, delete, and range operations.
- `table.py`: thin table wrapper over the tree.
- `db_manager.py`: manager of in-memory tables.
- Supporting files for brute-force and benchmarking.

This baseline had no explicit on-disk heap pages, WAL, transaction staging, crash replay, foreign key actions, or query planner behavior.

B. Final State (Module A Engine)

The final code in `Module-A/database` expands the design into a full mini-engine with these additions:

- `heapfile.py` for fixed-width binary storage.
- `wal.py` for write-ahead log entries with status transitions.
- `transactions.py` for BEGIN/COMMIT/ROLLBACK orchestration across tables.
- `column.py` and schema-level typing/default validation.
- `foreignkey.py` with RESTRICT/CASCADE/SET NULL/SET DEFAULT behavior.
- `secondaryindex.py` for composite secondary index entries.
- `conditions.py` + `query.py` for boolean expression trees and indexed candidate pruning.
- `join.py` for indexed join and nested-loop fallback plans.
- Usage scripts under `database/Usage/` with purpose-oriented names covering feature demonstrations and stress scenarios.
- Explicit assignment-facing test suite: `database/Tests/module_a_video_testcases.py`.

Only three Assignment 2 files were directly modified in place (`bplustree.py`, `table.py`, `db_manager.py`), while

all major transaction/recovery features were introduced in new modules.

IV. Core Architecture

A. Storage Layers

Each relation is represented by one `Table` object, but this object is intentionally built as a layered system rather than a single combined structure. The reason is that the assignment asks us to satisfy multiple goals at once: efficient B+ tree access, transaction safety, crash recovery, persistence across restarts, and correctness under concurrent operations. Trying to solve all of these in one layer makes the design fragile; separating responsibilities makes the system auditable and predictable.

The heap file is used as the durable row store because it gives deterministic slot-based addressing. This means every row can be located and rewritten directly, and the system does not depend on volatile in-memory state for final truth. The WAL exists to enforce write-ahead discipline: before any durable row mutation, the intent is journaled. That choice is the main protection against crash-time ambiguity. If a process stops between two operations, replay logic can reconstruct what must be applied and what must be ignored.

The primary B+ tree remains central and maps $pk \rightarrow slot$ so indexed access is fast and remains faithful to the assignment's B+ tree requirement. Storing slot references in the tree (instead of full row payloads) keeps the index lightweight and rebuildable from durable heap data. Metadata is split into a JSON artifact because schema and allocator counters evolve differently from raw row bytes, and this separation simplifies safe updates. Finally, the per-table lock in auto-commit mode exists to serialize write windows and prevent race-driven interleavings that could otherwise produce inconsistent partial state.

This decomposition was not added for style; it is the mechanism by which the design can recover deterministically after failure. On restart, the engine can replay WAL against heap, then rebuild indexes from recovered durable data. That order makes the system explainable and robust.

B. Record Layout in Heap File

The heap record format is fixed-width by design, and that choice is deeply connected to reliability. Each record starts with a tombstone marker so deletion can be modeled as a safe logical state change rather than immediate physical compaction. This avoids expensive rewrites and keeps slot ids stable, which matters because indexes and WAL entries refer to slots.

Each column stores an explicit null flag before encoded bytes. This prevents confusion between an actual value and a missing value, and it keeps null semantics stable even when binary defaults are zero-like. Deterministic width also means row offsets are computed directly: read and write paths do not need scanning or delimiter parsing, which reduces failure surfaces.

Type-aware encoding for `INT`, `FLOAT`, `BOOLEAN`, and bounded strings ensures that serialization and deserialization rules are consistent across normal execution and replay execution. This is crucial because WAL replay reuses the same packed bytes; if encoding were ambiguous, crash recovery could silently diverge from original writes.

V. WAL and Transaction Protocol

A. WAL Entry Semantics

The WAL stores operation type, slot id, transaction id, status byte, and packed record bytes. Status transitions are:

- `UNCOMMITTED`: staged or in-flight.
- `COMMITTED`: confirmed durable operation.
- `ROLLEDBACK`: intentionally discarded.

A pseudo transaction id `0` is reserved for auto-commit operations.

B. Auto-Commit Path

A single-table write follows:

1) Append WAL entry (uncommitted).
2) Write heap record (or tombstone for delete).
3) Mark WAL entry committed.
4) Update in-memory index structures.

This ordering implements write-ahead logging discipline.

C. Explicit Multi-Table Transaction Path

For explicit transactions:

1) `begin()` allocates a unique transaction id.
2) DML calls stage WAL entries only (heap untouched during staging).
3) `commit()` writes commit intent, applies pending entries to heap, then marks intent complete.
4) `rollback()` marks entries rolled back and rebuilds in-memory indexes from heap.

This design supports all-or-nothing behavior across enrolled tables and preserves a recovery signal (commit intent) for interrupted commit windows.

VI. Crash Recovery Design

At startup, each table executes a recovery pipeline:

1) Scan WAL and collect re-applicable entries.
2) Apply operations that are uncommitted auto-commit entries or belong to a transaction with an uncommitted commit-intent marker.
3) Skip committed and rolled-back entries already resolved.
4) Truncate corrupted/truncated tail fragments.
5) Rebuild in-memory indexes from heap.
6) Replace old WAL with a fresh session WAL.

This makes restart deterministic and idempotent at table granularity.

VII. Consistency Mechanisms Beyond ACID Skeleton

A. Schema Validation

`column.py` performs parse-time and runtime type checks, length constraints, nullability enforcement, and default-value validation. Errors are raised before storage mutation.

B. Foreign Keys and Cascades

`foreignkey.py` enforces parent-child constraints with action policies:

- `RESTRICT`
- `CASCADE`
- `SET NULL`
- `SET DEFAULT`

Referenced child lookups use secondary indexes for efficient fan-out updates/deletes.

C. Secondary Indexes

A secondary index stores composite keys $(column\_value, pk)$ in a B+ tree. This allows duplicate values and efficient equality/range scans while preserving deterministic retrieval of matching primary keys.

VIII. Query and Join Engine

A. Condition Trees and Filter Semantics

`conditions.py` models filters as expression trees (leaf, AND, OR, NOT) with operators including `IN`, `BETWEEN`, and null checks.

B. Index-Aware Candidate Generation

`query.py` extracts primary-key hints from safe AND branches and switches between indexed candidate fetching and full scan. Final filtering is always applied to guarantee semantic correctness.

C. Join Strategies

`join.py` chooses among:

- Index nested-loop (right join key is primary key).
- Secondary-index join (right join key has secondary index).
- Full nested-loop fallback.

LEFT join behavior is implemented by injecting null-valued right-side columns when no match is found.

IX. Micro-Level Implementation Details

This section captures fine-grained engineering choices that were intentionally crafted for correctness.

A. Table Lifecycle Internals

At runtime, a table is not only an in-memory object; it is a managed lifecycle over persistent and transient states. The persistent side consists of three artifacts: the heap file for row bytes, the WAL file for ordered mutation history, and metadata for schema plus allocator state. The transient side includes the primary B+ tree and optional secondary indexes used for fast routing.

The startup sequence is intentionally strict. WAL replay is performed first so unresolved operations are reconciled against durable storage. Only after this reconciliation are indexes rebuilt from heap. This ordering ensures that indexes never become an independent source of truth. If a crash happened earlier, stale in-memory index state is discarded and reconstructed from the recovered durable baseline.

B. Slot Allocation and Recycling

Insertion follows a two-step policy. The allocator first checks the free-list and reuses tombstoned slots when possible; only if no reusable slot exists does it allocate `next_slot` at the logical end. This policy is important for long-running workloads where frequent deletes and inserts would otherwise cause unnecessary file growth. Reuse also improves locality because frequently touched logical regions do not drift endlessly forward.

C. Row Packing Semantics

Row packing is deterministic and schema-ordered. Tombstone state, null flags, and encoded values are always laid out in the same byte pattern, and bounded strings are padded to fixed width. This makes row bytes replay-safe and eliminates ambiguity in partial-failure analysis. Because WAL stores full-row payloads, deterministic packing ensures replay logic applies exactly what was originally staged, not an interpretation that changes with runtime context.

D. WAL State Transitions at Byte Level

Each WAL entry includes operation code, slot id, transaction id, status byte, and data payload so that mutation history is both machine-readable and recovery-actionable. The state transition model is intentional: append as `UNCOMMITTED`, apply when execution reaches the safe stage, then finalize as `COMMITTED` or `ROLLEDBACK`. This explicit state machine is what allows replay logic to reason about interrupted sequences without guessing.

Status updates are implemented as targeted byte writes. This keeps finalization lightweight and avoids rewriting entire entries for one state flip, which reduces the chance that completion marking itself becomes a large interruption window.

E. Transactional Visibility Design

During explicit transactions, staged mutations are represented in WAL and selective in-memory index

state. Heap writes are deferred to commit. This allows transaction-time behavior to be controlled while preserving rollback simplicity, since rollback can invalidate staged WAL and rebuild index state from durable heap.

F. Foreign Key Enforcement Path

Foreign-key checks are executed before write completion. Parent existence checks use indexed paths. For delete/update actions on parent rows, referenced child rows are located through child-side secondary indexes so that cascades are targeted rather than full-table scans.

G. Query Evaluation Pipeline

The query executor intentionally separates optimization from truth evaluation. In the first stage, it narrows the candidate set using index hints only where logical safety is clear. In the second stage, it evaluates the full condition tree over candidates. This second stage is mandatory and is the correctness guardrail: even if pruning was conservative or incomplete, final predicate evaluation guarantees semantic accuracy.

H. Join Output Column Hygiene

Join merges preserve left-table column names directly. For right-table collisions, names are prefixed with table name. This avoids accidental overwrites in merged rows and keeps downstream filtering unambiguous.

I. Metadata Durability Discipline

Metadata is written through a temporary file and then atomically replaced. The reason is crash safety during bookkeeping updates: either the old metadata remains fully valid or the new metadata is fully installed. This prevents half-written allocator state from being accepted as valid and protects subsequent slot management and recovery decisions.

X. Methodology for Experimental Validation

A. Environment and Execution

All required scripts were executed using a Python virtual environment. Consolidated key outputs were captured in execution logs.

Scripts executed:

- `Usage/basic_table_transaction_acid_demo.py`
- `Usage/brutal_acid_transactions_and_persistence.py`
- `Usage/where_query_validation_and_stress.py`
- `Usage/deep_foreign_key_cascade_chain.py`
- `Usage/transactional_cascade_commit_rollback.py`
- `Usage/transactional_fk_smoke.py`
- `Usage/join_strategies_and_filters.py`
- `Usage/cascade_and_setnull_behaviour.py`
- `Usage/concurrency_isolation_numerical_integrity.py`
- `Usage/db_manager_platform_demo.py`
- `use_db.py`
- `Tests/module_a_video_testcases.py`

B. Observed Scenarios Covered

- Auto-commit inserts/updates/deletes and slot reuse.
- Cross-table transaction commit and rollback.
- Reload and persistence behavior after close/reopen.
- Constraint violation paths and guarded error handling.
- Deep cascading deletes and set-null propagation.
- Join strategy correctness with and without supporting indexes.
- Multi-thread stress inserts and high-contention partial updates.
- Numerically invariant transfer workloads under serialized critical sections.

XI. Execution Findings (Behavioral, Not Just Pass/Fail)

A. Atomicity Evidence

Transaction rollback scenarios demonstrate no partial visibility after failure:

- In one scenario, inserted records for the same transaction are absent post-rollback.
- In failure-injected transaction branches, staged changes are absent in final state after rollback.
- Cross-table transactional inserts appear together after commit and are jointly absent after rollback.

B. Consistency Evidence

Consistency is preserved through schema and referential constraints:

- Invalid primary-key updates are explicitly rejected.
- Foreign key checks prevent insertion of dangling child references.
- Cascade and set-null policies maintain referential closure after parent deletion.
- Deep-chain cascade scenarios leave expected surviving entities and clean null-linked children.

C. Isolation Evidence

Two levels of isolation behavior were observed:

- Auto-commit table writes are lock-serialized at table level to avoid interleaved corruption.
- Stress workload in `Usage/concurrency_isolation_numerical_` with 50 high-contention updates produced an aggregate flag sum of `50/50`, showing no lost column updates.

For threaded transfer simulations, global serialization around critical transfer windows preserved global invariants while allowing high operation volume.

D. Durability Evidence

Durability appears through repeated close/reopen and reload scenarios:

- Rows written and committed before close are available after reopening tables.
- Deleted rows remain absent across restart.
- Transaction-committed order rows persist and can be re-fetched after reload.

XII. Representative Output Snapshots

A. Numeric Invariant Under Concurrent Transfers

From stress execution output:

- Original system balance: `2000000`
- Final system balance: `2000000`

This indicates no net value creation or loss under concurrent transfer generation with serialized critical sections.

B. Contention Update Integrity

From the same run:

- Final aggregated flag sum: `50 / 50`

This confirms all intended partial updates to the same logical row were preserved.

C. Cascade and Set-Null Effects

Deep cascade scripts report expected structural outcomes such as:

- Parent deletions propagating to child deletes where `CASCADE` applies.
- Child references becoming null where `SET NULL` applies.
- Surviving rows retaining valid references after each action.

XIII. Detailed Experiment Walkthrough

This section records what each script was used for and what final system behavior was observed. The intent is to document the engineering narrative in detail instead of reducing the experiments to binary pass/fail labels.

A. Core Storage and Transaction Lifecycle

The `Usage/basic_table_transaction_acid_demo.py` script exercises the fundamental lifecycle of the table engine:

- Auto-commit insert and point lookup to validate baseline correctness of heap and B+ tree updates.
- Auto-commit update and delete to verify mutation semantics, slot recycling, and key visibility.
- Range scans and full-order traversal to validate in-order key iteration from B+ tree leaf chaining.
- Default value and null handling paths that ensure schema-level semantics are preserved in binary layout.
- Duplicate primary-key error path to show conflict prevention before storage mutation.
- Close/reload cycle confirming persistence and index rebuild correctness.
- Explicit multi-table transaction commit introducing rows in users and orders atomically.
- Explicit rollback and exception-driven rollback confirming staged entries are discarded correctly.
- Transactional update and delete commit path, followed by reload to prove durable final state.

Observed final state lines include stable post-reload records such as user score updates and missing deleted order ids, which demonstrates practical end-to-end behavior from WAL through heap to index rebuild.

B. Brutal Mixed-Mode Transaction Scenarios

The `Usage/brutal_acid_transactions_and_persistence.py` workflow focuses on heavy combinations of:

- Auto-commit DML sequences.
- Explicit transaction batches.
- Cross-table atomic commit requirements.
- Reload behavior after mixed write patterns.
- Mini stress patterns where auto-commit and transaction windows interleave over time.

The important engineering takeaway is that no residual partial updates were observed after rollback-oriented branches, while committed branches were visible after reload.

C. Query Algebra and Condition Semantics

The `Usage/where_query_validation_and_stress.py` script verifies condition-tree semantics and fluent builder behavior:

- Leaf operators and combinators (AND/OR/NOT).
- Count/first semantics on filtered subsets.
- Validation and guarded error paths under malformed conditions.
- Transaction visibility interactions with where-clause filtering.
- Reload verification after query-heavy mutation phases.

This validates that candidate pruning and final condition evaluation remain logically aligned with intended query semantics.

D. Deep Referential Cascading

The deep-cascade scripts construct multi-level dependency chains (e.g., country/state/city/university/student/enrollment style structures and department/employee/-task/asset structures). The experiments then execute parent deletions and observe:

- Recursive deletion propagation for `CASCADE` relationships.
- Nullification for `SET NULL` relationships.
- Expected survivors in non-target branches.
- Referentially valid terminal state after each operation.

These scenarios are critical because they combine consistency logic with mutation ordering across multiple tables.

E. Transactional Cascade Correctness

These scripts focus on cascade outcomes under transactional control:

- Uncommitted delete effects are visible in transaction-time logic.
- Rollback restores parent and child relations to pre-transaction state.
- Commit permanently applies parent and child deletions.

This directly supports the assignment requirement that partial outcomes must not remain after aborted execution.

F. Join Strategy Validation

Join behavior was exercised along three implementation routes:

- PK-based indexed nested-loop joins.
- Secondary-index accelerated joins.
- Full nested-loop fallback for unindexed predicates.
- Additional where-clause filtering over join results.

Observed outputs included expected row cardinalities and correctly merged key namespaces, including left-join behavior for unmatched categories.

G. Concurrency and Numerical Safety

The most isolation-sensitive script performs:

- Twenty-thread bulk insertion waves.
- Fifty-thread high-contention partial updates on the same logical row.
- Large randomized transfer sequences with serialized transaction windows.

The two key numeric outcomes were retained exactly:

- Aggregated contention-update flags: `50/50`.
- Global transfer balance invariant: `2000000` before and after.

These observations provide concrete evidence that write serialization and transaction windows preserved numeric consistency.

H. Manager-Facade End-to-End Usability

These scripts demonstrate the full platform from a consumer perspective: schema declaration, table creation, foreign key setup, transaction usage through manager-level APIs, joins, and cascade operations. The final behavior showed successful transaction commits, high-value order retrieval through joins, and correct cascade cleanup after deleting a parent user/department.

From a software engineering perspective, these scripts validate that the low-level storage and recovery mechanisms are not isolated internals: they are successfully exposed through a coherent high-level interface.

XIV. Requirement Traceability Matrix

XV. Assignment-Facing Video Testcases (Explicit 5)

To make the demo output understandable to a reviewer (not just pass/fail lines), `Tests/module_a_video_testcases.py` runs five explicit scenarios and prints initial state, operation, and final observed state for each.

XVI. Engineering Rationale and Internal Design Decisions

This section explains why specific implementation choices were made, including trade-offs relative to simpler alternatives.

TABLE I
Assignment Requirement Coverage in Final Module A

Requirement	Mechanism	Evidence
Atomicity	Staged WAL + commit/rollback coordinator	Rollback scenarios in `Usage/basic_table_transaction_acid_dem` and `Tests/module_a_video_testcases.py` show no partial persistence
Consistency	Schema checks + FK actions + controlled updates	Error-path scenarios and cascade chains in `Usage/deep_foreign_key_cascade_chain.p` `Usage/cascade_and_setnull_behaviour.py` and testcase 3 in `Tests/module_a_video_testcases.py`
Isolation	Table lock for auto-commit + serialized transfer critical sections	High-contention and transfer scenarios in `Usage/concurrency_isolation_numerical_` and testcase 4 in `Tests/module_a_video_testcases.py`
Durability	Heap persistence + startup replay + reload tests	Reopen/reload outputs in `Usage/basic_table_transaction_acid_dem` and testcase 5 in `Tests/module_a_video_testcases.py`
Multi-relation txn	Transaction across enrolled table set	Cross-table insert/rollback examples in brutal transaction script and testcases 2 to 3 in video tests
Crash handling	WAL replay and commit-intent semantics	Recovery workflow in `wal.py` and table startup logic

TABLE II
Five Explicit Testcases Mapped to Assignment Requirements

ID	Scenario	Requirement	Visible output shown
TC1	Basic table lifecycle demo (insert/update/delete/read)	Table operation baseline	Step-by-step initialization, mutation, and final verification output
TC2	Mid-transaction failure across users/products/orders	Atomicity	Unchanged balance/stock and missing order after forced failure
TC3	Successful 3-table order transaction + invalid operation checks	Atomicity + Consistency + Multi-relation txn	Commit success plus rejection messages for invalid operations/FK violations
TC4	Concurrent transfer workload with serialized transaction windows	Isolation	Initial total balance equals final total; no negative balances
TC5	Restart recovery with committed, failed, and orphan staged work	Durability + Crash handling	Committed row persists; failed/orphan rows absent after reload

A. Why Heap + WAL + In-Memory Index Instead of Direct Tree Serialization

A direct approach could have been to serialize full B+ tree nodes to disk for every operation. However, for this assignment, that would increase implementation complexity in crash scenarios because in-flight node splits/merges can span multiple pages and pointer updates. The selected

architecture keeps the durable row representation in a fixed-width heap file and uses the in-memory B+ tree as a fast routing/index layer rebuilt from durable state at startup.

This split yields three practical benefits:

- Row persistence is simple and deterministic (slot-addressed writes).
- WAL recovery replay can be expressed as operation-level idempotent record writes/deletes.
- Index invariants are reconstructed from committed row state, reducing corruption risk after interruptions.

The trade-off is startup rebuild time proportional to table size, which is acceptable for assignment-scale datasets and greatly simplifies correctness arguments.

B. Why Stage Transaction Operations in WAL Before Heap Apply

The transaction path writes only to WAL during staging and delays heap mutation until commit. This design prevents partial heap updates for uncommitted work and naturally supports rollback by status flipping plus index rebuild. In other words, rollback does not need physical undo in the heap because uncommitted transaction rows were never persisted there.

An alternative approach (write-through with compensating undo entries) was avoided because it requires more elaborate undo logging and careful ordering guarantees to remain safe under process termination.

C. Commit-Intent Marker and Failure Window Closure

A key detail is the commit-intent WAL entry. Without this marker, a crash between "start commit" and "all entries marked committed" can leave ambiguous intent. With commit-intent persisted first, recovery can determine that the transaction was in its commit phase and should be completed by replaying relevant uncommitted entries. This closes the classic uncertainty window where some data may have been applied but status bits are incomplete.

D. Isolation Strategy Selection

The assignment permits basic locking or serialized execution. The implementation therefore uses:

- Per-table lock for auto-commit operations.
- Serialized critical sections in stress transfer workflows.

This is intentionally simpler than full lock manager + deadlock detection + multi-version concurrency control. The practical result is predictable correctness under concurrent pressure with manageable implementation complexity in Python.

E. Foreign Key Actions Coupled with Secondary Indexes

Cascade operations are potentially expensive if every parent delete requires full child-table scans. Coupling foreign key tracking with per-column secondary indexes allows child primary-key discovery in logarithmic/range traversal costs. This is a crucial design decision for deep cascade chains, where repeated full scans would otherwise dominate runtime and increase contention windows.

F. Query Planner Conservatism for Correctness

In `query.py`, primary-key hints are extracted only from safe AND branches. OR and NOT branches do not drive candidate pruning because unsafe pruning could silently drop valid rows. This conservative approach prioritizes semantic correctness over aggressive optimization. For educational and validation-focused assignments, this is a better trade than implementing complex logical rewrites that may introduce subtle bugs.

G. Deletion via Tombstones and Free List Reuse

Deleting heap records by tombstone (instead of immediate physical compaction) avoids expensive file rewrites and preserves stable slot addressing for WAL replay semantics. A free-list allocator then recycles deleted slots for new records. This design keeps write costs bounded while preserving deterministic slot lifecycle tracking in metadata.

H. Failure Modes Considered During Design

The implementation explicitly addresses the following failure classes:

- Crash after WAL append but before heap write.
- Crash after heap write but before WAL status flip.
- Crash during commit of multi-entry transaction.
- Truncated WAL tail due to interrupted writes.
- Process restart with stale in-memory index state.

For each class, startup replay and index rebuild paths provide a deterministic convergence to a valid state.

I. Pragmatic Scope Boundaries

The system does not claim complete industrial SQL engine behavior. Instead, it targets the assignment's reliability and correctness goals with clear and inspectable mechanisms. This scoped approach is a strength for learning-oriented database engineering because each guarantee can be traced from code path to observed runtime behavior.

XVII. What Was Tested and Final Outcome Summary

The testing effort focused on behavioral validation rather than test-count reporting. In practical terms, the following were validated end-to-end:

- Data mutation correctness under normal auto-commit operation.
- Correct rollback of multi-step and exception-driven transactions.
- Referential integrity through deletion/update cascades and nullification.
- Query/filter correctness for complex conditions and chained combinators.
- Join correctness across primary-key, secondary-index, and fallback plans.

- Robustness under multithreaded pressure and numerically sensitive workloads.
- Persistence and logical state recovery after close/reopen cycles.

Final outcome: the final Module A implementation behaves as a structured ACID-oriented mini-engine with explicit mechanisms for recovery, consistency enforcement, and practical isolation controls suitable for the assignment scope.

XVIII. Limitations and Future Improvements

- Transactional isolation is practical but not a full MVCC or strict serializable scheduler.
- WAL maintenance currently resets per startup cycle; archival log retention can be added.
- Fine-grained lock management and deadlock detection are not yet modeled.
- Cost-based optimizer choices for join/query paths can be expanded.
- Fuzz-style crash injection at instruction-level granularity can further strengthen durability claims.

XIX. Conclusion

This Module A implementation transforms a pedagogical in-memory B+ tree table layer into a substantially stronger database kernel with persistent heap storage, WAL-backed mutation ordering, transaction staging and commit orchestration, restart recovery, referential integrity actions, and index-aware query/join behavior. The executed scenarios demonstrate that correctness is not accidental: it is designed through explicit data layout, operation sequencing, and failure-aware state transitions.

By mapping each required assignment capability to concrete implementation paths and runtime observations, the work establishes that the system now behaves as a coherent ACID-oriented engine rather than a standalone data structure wrapper.

Appendix A
Appendix A: What We Crafted and Why

This appendix explains every Python file in the newer database folder in simple terms. The goal is to show how each file contributes to the final transactional database engine.

A. Core System Files

1. `__init__.py`
Marks the directory as a Python package so modules can import each other cleanly.
2. `bplustree.py`
Implements the primary in-memory index structure. It handles search, insert, delete, and range traversal, and supports composite keys used by secondary indexes.
3. `heapfile.py`
Implements durable fixed-width binary storage. It packs/unpacks rows, manages null flags and tombstones, and performs slot-based reads/writes.
4. `wal.py`
Implements write-ahead logging. It records operations before heap writes, tracks commit/rollback status, and supports replay after crash/restart.
5. `transactions.py`
Implements explicit multi-table transactions with begin, commit, rollback, and staged WAL application.
6. `table.py`
This is the core table engine. It combines schema validation, WAL ordering, heap mutation, index maintenance, foreign-key checks, and recovery startup behavior.
7. `db_manager.py`
Provides a high-level API to create/drop/get tables, start transactions, and run joins through one central manager object.

B. Schema, Constraint, and Index Files

8. `column.py`
Defines column types, default/null rules, and value validation. It ensures bad data is rejected before persistent writes.
9. `foreignkey.py`
Defines referential integrity behavior for parent-child links, including RESTRICT, CASCADE, SET NULL, and SET DEFAULT actions.
10. `secondaryindex.py`
Implements non-primary indexes using composite keys $(value, pk)$ over B+ tree, enabling efficient lookup of duplicate secondary values.

C. Query and Join Files

11. `conditions.py`
Defines condition trees (leaf, AND, OR, NOT) and operator logic for filtering rows.
12. `query.py`
Implements fluent filtering and candidate selection. It uses index hints when safe and always applies full-condition checks for correctness.
13. `join.py`
Implements join execution with automatic strategy choice: primary-index join, secondary-index join, or nested-loop fallback.

D. Evaluation and Utility Files

14. `bruteforce.py`
Contains a simpler baseline style implementation used for comparison and sanity checks.
15. `performance.py`
Provides benchmarking and analysis helpers for measuring operation behavior and comparative performance.
16. `run_eval.py`
Utility script for cleanup/evaluation setup before running experiments.
17. `python_test.py`
General Python-level check script used during development validation.

E. Automated Test Files

18. `test_bplustree.py`
Focused tests for B+ tree correctness: insert/search ordering, range behavior, delete behavior, and randomized operations.
19. `test_join_secondary.py`
Focused tests for join behavior and secondary-index-assisted retrieval.

F. Usage Scenario Files (Demonstration Suite)

20. `Usage/basic_table_transaction_acid_demo.py`
Main functional walkthrough covering CRUD, transactions, and persistence.
21. `Usage/brutal_acid_transactions_and_persistence.py`
Brutal mixed workload scenarios with auto-commit + explicit transactions.
22. `Usage/where_query_validation_and_stress.py`
Condition/query stress scenarios with combinator-heavy filtering.
23. `Usage/deep_foreign_key_cascade_chain.py`
Deep cascading referential scenarios over long dependency chains.
24. `Usage/transactional_cascade_commit_rollback.py`
Transactional cascade behavior with rollback vs commit comparison.
25. `Usage/transactional_fk_smoke.py`
Compact cascade sanity scenario for quick verification.
26. `Usage/join_strategies_and_filters.py`
Join strategy scenarios across indexed and unindexed paths.
27. `Usage/cascade_and_setnull_behaviour.py`
Combined CASCADE and SET NULL behavior in richer linked schemas.
28. `Usage/concurrency_isolation_numerical_integrity.py`
Concurrency stress scenarios: threaded inserts, contention updates, and numeric invariant checks.
29. `Usage/db_manager_platform_demo.py`
Application-style usage from manager API perspective: setup, transaction, join, and cascade flow.
30. `use_db.py`
Comprehensive end-to-end demo script combining multiple features in one narrative run.
31. `Tests/module_a_video_testcases.py`
Five explicit assignment-facing testcases with readable before/after outputs for demo recording.

G. Why This Full File Set Matters

Together, these files cover all layers required by the assignment: storage, indexing, transactions, crash recovery, constraints, query processing, joins, stress scenarios, and validation scripts. That is why the newer database folder represents a complete engine evolution rather than isolated feature additions.

Appendix B

Appendix B: How to Read the Test Outcomes

This appendix explains each usage script in direct and simple terms. Here, "Usage 1" corresponds to `Usage/basic_table_transaction_acid_demo.py`, and "Usage 2 to 10" correspond to the remaining scripts under `Usage/`.

A. Usage 1

File: `Usage/basic_table_transaction_acid_demo.py`.

What we tested: Core database behavior end-to-end: insert, update, delete, range query, default values, transaction commit, transaction rollback, and reload from disk.

What this means: This usage verifies the base ACID flow for day-to-day operations. It shows that regular writes work, rollbacks remove partial state, and committed data survives reopen.

B. Usage 2

File: `Usage/brutal_acid_transactions_and_persistence.py`.

What we tested: More aggressive mixed workload with auto-commit plus explicit transaction blocks, including cross-table atomic behavior and reload checks.

What this means: This usage confirms that the system remains correct when operation styles are mixed and when multiple tables are involved in the same logical action.

C. Usage 3

File: `Usage/where_query_validation_and_stress.py`.

What we tested: Condition evaluation and query chaining: AND/OR/NOT combinations, count/first behavior, and validation/error paths.

What this means: This usage validates that query semantics are correct and predictable, even for complex filters, and that invalid query paths are handled safely.

D. Usage 4

File: `Usage/deep_foreign_key_cascade_chain.py`.

What we tested: Deep cascading relations across many linked tables, especially delete propagation in long dependency chains.

What this means: This usage shows referential integrity is not shallow. Parent-level operations propagate correctly through deeper levels without leaving broken links.

E. Usage 5

File: `Usage/transactional_cascade_commit_rollback.py`.

What we tested: Transactional cascading behavior with explicit rollback and commit comparison on parent-child tables.

What this means: This usage demonstrates that cascade effects are transaction-aware: they disappear on rollback and persist on commit.

F. Usage 6

File: `Usage/transactional_fk_smoke.py`.

What we tested: A compact focused cascade scenario to validate a clean terminal state after deletion.

What this means: This usage gives a minimal sanity check for referential action correctness in a smaller, easy-to-verify setup.

G. Usage 7

File: `Usage/join_strategies_and_filters.py`.

What we tested: Join behavior under three modes: primary-key indexed join, secondary-index join, and nested-loop fallback, plus filtering with WHERE.

What this means: This usage confirms join correctness independent of strategy. If indexing is available, faster routes are used; if not, fallback still returns correct results.

H. Usage 8

File: `Usage/cascade_and_setnull_behaviour.py`.

What we tested: Combined CASCADE and SET NULL behavior in a richer practical schema (for example, department-employee-task-asset style links).

What this means: This usage shows that different referential policies can coexist and still produce a consistent final state after parent operations.

I. Usage 9

File: `Usage/concurrency_isolation_numerical_integrity.py`.

What we tested: Concurrency stress: multithread inserts, high-contention updates on shared rows, and transfer-style operations with numeric invariants.

What this means: This usage validates practical isolation under pressure. The important outcome is invariant preservation (no lost updates and no net balance drift).

J. Usage 10

File: `Usage/db_manager_platform_demo.py`.

What we tested: Full platform demonstration from the manager interface: table setup, foreign keys, transactions, joins, and cascading delete effects.

What this means: This usage shows that the engine is usable as an integrated system, not only as low-level internals. Core guarantees remain visible at application-facing API level.

K. Overall Interpretation Across Usage 1 to 10

Taken together, these ten usages map directly to assignment intent: ACID behavior, cross-table correctness, recovery persistence, referential integrity, query/join correctness, and concurrent robustness. The combined outcomes indicate that the system behaves consistently as a transactional database engine rather than a collection of isolated features.

Appendix C
Contribution Table

TABLE III
Team Contribution Summary

Member	Contribution
Tejas Lohia	Module A, video, report
Umang Shikarvar	Module A and B designing
Zainab Kapadia	Module B, video
Siddharth Rajandekar	Module B, report
Rishabh Jogani	-